Bugs in Modern LLM Agent Frameworks: An Empirical Study

Xinxue Zhu
Nantong University
Nantong, China
804134381@qq.com

Jiacong Wu
Nanjing University
Nanjing, China
776a6301@gmail.com

Xiaoyu Zhang*
Nanyang Technological University
Singapore, Singapore
xiaoyu.zhang@ntu.edu.sg

Tianlin Li
Beihang University
Beijing, China
tianlin001@buaa.edu.cn

Yanzhou Mu
Nanjing University
Nanjing, China
602022320006@smail.nju.edu.cn

Juan Zhai
University of Massachusetts Amherst
Amherst, USA
juanzhai@umass.edu

Chao Shen
Xi'an Jiaotong University
Xi'an, China
chaoshen@mail.xjtu.edu.cn

Chunrong Fang
Nanjing University
Nanjing, China
fangchunrong@nju.edu.cn

Yang Liu
Nanyang Technological University
Singapore, Singapore
yangliu@ntu.edu.sg

Abstract

LLM agents have been widely adopted in real-world applications, relying on agent frameworks for lifecycle execution and multi-agent coordination. As these systems scale, understanding bugs in the underlying agent frameworks becomes critical. However, existing work mainly focuses on agent-level failures, overlooking framework-level bugs. To address this gap, we conduct an empirical study of 998 issue reports from CrewAI and LangGraph, and construct a taxonomy of 15 root causes and seven observable symptoms across five agent lifecycle stages: 'Agent Initialization', 'Perception', 'Self-Action', 'Mutual Interaction' and 'Evolution'. Our findings show that agent framework bugs mainly arise from 'API Misuse', 'API Incompatibility', and 'Documentation Desync', largely concentrated in the 'Self-Action' stage. Symptoms typically appear as 'Functional Error', 'Crash', and 'Build Failure', reflecting disruptions to task progression and control flow.

CCS Concepts

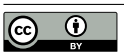
• **Software and its engineering → Software testing and debugging**.

Keywords

LLM Agent Framework, Bug Taxonomy, Empirical Study

ACM Reference Format:
Xinxue Zhu, Jiacong Wu, Xiaoyu Zhang, Tianlin Li, Yanzhou Mu, Juan Zhai, Chao Shen, Chunrong Fang, and Yang Liu. 2026. Bugs in Modern LLM Agent Frameworks: An Empirical Study. In *34th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (FSE Companion '26), July 5–9, 2026, Montreal, QC, Canada.* ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/3803437.3805536

*Xiaoyu Zhang is the corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
FSE Companion '26, Montreal, QC, Canada
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2636-1/26/07
https://doi.org/10.1145/3803437.3805536

1 Introduction

Large language models (LLMs) achieve rapid progress in real-world industry applications like intelligent Q&A [6], automated software engineering [4], and multimodal autonomous driving [5, 7]. As LLM capabilities grow, single-call usage patterns are often insufficient for long-horizon, tool-driven tasks, motivating the agent paradigm that integrates models with memory, tools, and control logic. To support the development of LLM agents, researchers introduce LLM agent frameworks (e.g., LangGraph [1]) that help users define task workflows, manage agent state, coordinate actions and tools, and integrate agents into larger software systems. When bugs arise in LLM agent frameworks, they can propagate to upper-layer systems and amplify their impact, leading to incorrect execution, resource misuse, and security risks. Such bugs in LLM infrastructure may propagate throughout the LLM software supply chain [8] and pose a serious security threat to agent-based software systems. Therefore, there is an urgent need to study and understand their observable symptoms and underlying root causes to strengthen the quality of the LLM software supply chain. However, existing work [3, 10] primarily investigates agent-level failures in reasoning, planning, and action processes, often through behavioral analysis or benchmark evaluation. These studies enhance our understanding of agent behaviors and model limitations, but largely overlook bugs rooted in the underlying agent frameworks. While a recent study [9] provides an initial characterization of bugs in agent libraries by mapping pull requests to static components of libraries (e.g., data preprocessing), this approach overlooks the dynamic execution and temporal lifecycles of agents. Consequently, how LLM agent framework bugs manifest across the dynamic agent lifecycle remains largely unexplored.

To address this gap, in this paper, we collect and manually analyze 998 issue reports from two representative agent frameworks, CrewAI [2] and LangGraph [1], to identify the root causes and symptoms of diverse agent framework bugs. We map these findings onto five agent lifecycle stages (i.e., 'Agent Initialization', 'Perception', 'Self-Action', 'Mutual Interaction', and 'Evolution') and construct a taxonomy comprising 15 root causes and seven symptoms. This lifecycle-oriented taxonomy clarifies where bugs arise and how

framework-level issues propagate during execution. We further formulate two research questions (RQs) to examine their root causes and observable symptoms, as shown in the following.

• **RQ1: What are the root causes of Agent framework bugs?** Identifying root causes is essential for explaining why bugs repeatedly occur and for revealing dominant bug sources within agent frameworks. From 998 issue reports, we identify 15 root cause categories, with 'API Misuse' (32.97%) and 'API Incompatibility' (22.34%) accounting for over 55% of all cases. Most root causes concentrate in the 'Self-Action' stage, indicating that mechanisms related to execution semantics are the primary source of framework bugs.

• **RQ2: What are the symptoms of Agent framework bugs?** Characterizing symptoms clarifies how LLM agent framework bugs surface during execution and what breakdown patterns users ultimately encounter. We identify seven symptom categories, which also predominantly cluster in the 'Self-Action' stage. The categories of 'Crash', 'Functional Error', and 'Build Failure' account for the majority of observable disruptions, suggesting that framework bugs mainly manifest as breakdowns in lifecycle progression rather than isolated interface bugs.

Overall, the main contributions of this work are as follows.

- **Innovative Perspective.** We introduce a lifecycle-oriented perspective for analyzing LLM agent framework bugs, organizing bugs across key stages of 'Agent Initialization', 'Perception', 'Self-Action', 'Mutual Interaction', and 'Evolution'. This perspective provides a structured foundation for understanding where and how framework-level bugs arise during agent execution.
- **Empirical Study and Key Findings.** Based on 998 issue reports from two LLM agent frameworks, our study constructs a taxonomy of 15 root causes and seven symptoms, and reveals their non-trivial distributions and correlations across lifecycle stages, highlighting execution-semantics mechanisms as the dominant source of bugs.
- **Reproducible Artifacts.** We release our curated dataset, taxonomy definitions, and analysis scripts to facilitate replication and future research on LLM agent frameworks.

2 Methodology

To investigate bugs in agent frameworks, we adopt a three-step methodology, as shown in Figure 1. **(1) Issue Collection.** We gather issue reports from open-source agent framework communities. **(2) Bug Filtering.** We apply predefined criteria to remove irrelevant or duplicate reports, ensuring data reliability. **(3) Individual Labeling.** The two authors first construct an initial taxonomy based on 100 samples and then apply it to large-scale annotation. Finally, we construct a lifecycle-oriented taxonomy that maps root causes and symptoms to the five agent lifecycle stages, providing a structured foundation for analysis.

2.1 Issue Collection

To study bugs in agent frameworks, we select two representative and widely used open source frameworks, namely crewAI [2] and langgraph [1], as our research subjects. Our selection is motivated by both their practical influence and their architectural complementarity. On the one hand, both frameworks have gained substantial traction in the open source community and have been repeatedly considered in recent comparative and benchmark-oriented studies of agent systems. On the other hand, they capture two important design paradigms in current LLM agent frameworks. CrewAI emphasizes high-level, role-based abstractions for multi-agent collaboration, whereas LangGraph provides a more general-purpose, graph-based orchestration model centered on explicit state management, durable execution, and controllable workflows. By examining these two frameworks together, we are able to cover both collaborative multi-agent development and general stateful agent orchestration, thereby capturing a broad portion of the contemporary agent frameworks design space. Moreover, their active GitHub repositories and large user communities (these two frameworks receive 68,500 stars and 55,000 users) provide a rich and credible source of issue reports for studying real-world framework bugs.

We adopt a full data scraping strategy that includes both open and closed issues to reduce sampling bias and ensure comprehensive coverage of framework bugs. We collect a total of 2,773 original issue reports from the two selected frameworks, including 1,660 issues from CrewAI and 1,113 issues from LangGraph, spanning the period from December 7, 2023, to January 10, 2026. For each issue report, we retrieve and extract its core content for subsequent bug analysis, including title, labels, content, and comments. These elements together form the basis for our empirical analysis.

2.2 Bug filtering

We adopt a two-stage filtering process to refine the collected issue reports and ensure analytical relevance, as follows.

Stage 1: Label Filtering. We first leverage GitHub labels to perform preliminary filtering. For each issue, we check whether maintainers assign the label 'bug'. If the issue contains this label, we retain it for further analysis; otherwise, we exclude it. This step narrows the dataset to issues that repository maintainers or contributors explicitly marked as potential bugs and reduces irrelevant entries at an early stage. After this stage, 1,010 issue reports remain.

Stage 2: Manual Inspection. We then conduct a manual inspection to eliminate mislabeled or irrelevant reports. Two researchers independently examine the title, issue description, and associated comments of each pre-screened issue and evaluate them against our operational definition of an agent framework bug. During this stage, we exclude three main categories of issue reports, including ❶ non-functional textual errors such as documentation typos, ❷ incomplete documentation requests or usage questions, ❸ invalid or infrastructure-related issues that do not involve framework logic. After this two-stage filtering process, we obtain a refined dataset of 998 issues that directly relate to agent framework bugs and form the basis for subsequent analysis and classification.

2.3 Individual Labeling

Two authors serve as the annotators to label the issue reports. The overall process consists of two key stages: initial taxonomy construction and large-scale annotation.

Stage I: Initial Taxonomy Construction. To ensure labeling reliability and establish a principled classification scheme, we randomly sample 100 issue reports from the refined dataset as a representative subset for initial analysis. Two authors, each with at least

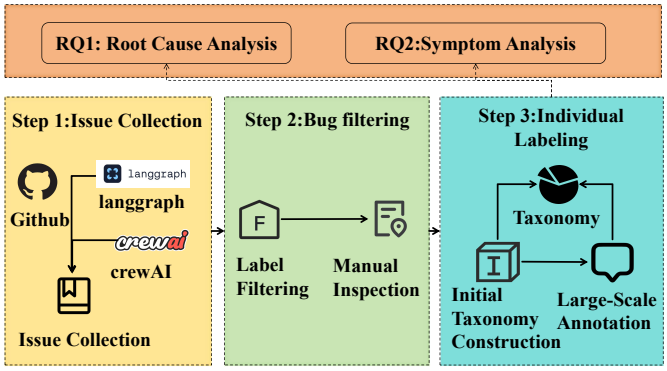


Figure 1: Overview of our Three-Step Methodology

one year of experience in software engineering research on LLM-based systems, independently examine each sampled report and perform three tasks: identifying the primary framework component involved, inferring the underlying root causes of the abnormal behavior, and documenting the symptoms manifested during execution. For inconsistent cases, the two authors discuss the reports and reach consensus through online meetings. From this process, we derive an initial taxonomy consisting of 15 root cause categories and seven symptom categories.

Stage II: Large-Scale Annotation. With the initial taxonomy established, two annotators apply it to the remaining issue reports for large-scale labeling. For each issue, annotators follow the same process used in Stage I. Then, annotators record category assignments directly using the predefined taxonomy and document brief justifications when classification requires interpretation. During this process, a new category is introduced only if meeting issues whose root causes or symptoms do not clearly fit any existing category.

Finally, we obtain a taxonomy consisting of 15 root cause categories and seven symptom categories from 998 issue reports.

3 Result Analysis

3.1 RQ1: Root Causes

Based on the classification of 998 issue reports, we identify 15 root cause categories. 'API Misuse' and 'API Incompatibility' dominate the distribution, together accounting for over half of the cases, highlighting the central role of interface-related problems in agent framework bugs. Other categories, such as 'Frontend–Backend Mismatch' and 'Console Interaction Bug', occur far less frequently. These results indicate that most bugs stem from execution semantics and API evolution rather than isolated infrastructure bugs (Figure 2).

•*R1.API Incompatibility (22.34%).* This root cause emerges when changes in agent frameworks APIs alter execution semantics across planning, tool invocation, or multi-step workflows, thereby breaking assumptions embedded in agent logic rather than merely causing surface-level version conflicts.

• *R2.API Misuse (32.97%).* This root cause occurs when developers misunderstand the agent framework's execution abstractions, control-flow design, or lifecycle semantics, leading to incorrect orchestration of tools, memory, or agent interactions.

• *R3.Configuration Misalignment (5.31%).* It appears when the agent frameworks expect specific runtime configurations (e.g., model backends, tool registries), but the actual environment violates these assumptions and disrupts agent execution semantics.

• *R4.Telemetry Malfunction (3.21%).* This root cause arises when tracing, logging, or observability components embedded in the agent frameworks interfere with execution flow or fail to correctly capture agent state transitions.

• *R5.Streaming Instability (6.71%).* This root cause occurs when the framework's streaming mechanisms, which coordinate incremental model outputs and tool responses, lose synchronization or terminate prematurely, breaking workflow continuity.

• *R6.Serialization Error (5.31%).* This root cause appears when the framework fails to correctly encode or decode structured execution artifacts such as agent state, intermediate plans, or tool-call messages, leading to corrupted execution context.

• *R7.Execution Isolation (2.00%).* This root cause emerges when sandboxing policies or security constraints enforced by the agent frameworks conflict with intended tool execution or cross-agent coordination semantics.

• *R8.Memory Persistence Bug (3.01%).* This root cause occurs when the framework's memory management mechanisms fail to maintain consistent long-term state, embedding indices, or retrieval mappings across agent iterations.

• *R9.Console Interaction Bug (1.50%).* This root cause arises when the framework assumes interactive execution patterns that do not align with the actual runtime context, thereby disrupting command handling or workflow triggering.

• *R10.Documentation Desync (7.52%).* This root cause appears when inconsistencies between documented abstractions and actual framework behavior mislead developers about lifecycle semantics, API contracts, or execution guarantees.

• *R11.Knowledge Transmission Bug (2.10%).* It emerges in multi-agent settings when the framework fails to correctly propagate context, task specifications, or role-specific information across agents.

• *R12.Frontend-Backend Mismatch (0.70%).* This root cause occurs when inconsistencies between monitoring interfaces and backend execution logic lead to misleading state visualization or incorrect control signals during agent operation.

• *R13.Concurrency Behavior Confusion (2.00%).* This root cause arises when frameworks' concurrency model, e.g., asynchronous graph execution or task scheduling semantics, creates implicit behaviors that developers misinterpret during agent orchestration.

• *R14.Dependency Environment Inconsistency (3.51%).* This root cause appears when the framework's required software stack conflicts with the surrounding ecosystem, preventing the correct initialization of agent runtimes, model backends, or tool integrations.

• *R15.Other (1.80%).* This category includes low-frequency root causes that do not fit existing groups but still expose implicit design assumptions in agent frameworks, such as hidden dependencies on file systems, operating systems, or execution boundaries.

Analysis on Agent Lifecycle Distribution. We further examine how root causes distribute across the five lifecycle stages. Although some types span multiple stages, clear concentration patterns emerge.

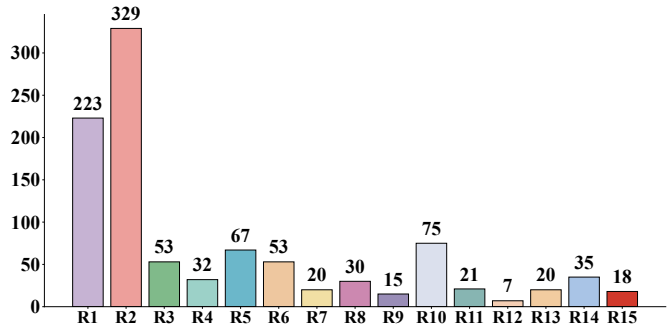


Figure 2: Root Cause Distribution of Agent Framework Bugs

The 'Agent Initialization' stage mainly involves 'Documentation Desync' (accounting for 30.56% of this stage), 'API Misuse' (accounting for 22.22% of this stage), and 'API Incompatibility' (accounting for 16.67% of this stage), with fewer cases of 'Dependency Environment Inconsistency', 'Configuration Misalignment', and 'Execution Isolation'. These issues block proper setup, capability registration, or parameter configuration, creating flawed execution foundations.

Bugs in the 'Perception' stage mainly involve interface misuse and configuration inconsistency. 'API Misuse' (accounting for 44.44% of this stage) and 'Documentation Desync' (accounting for 16.67% of this stage) lead to incorrect calls or outdated assumptions, while 'Configuration Misalignment' and 'Frontend–Backend Mismatch' disrupt environmental alignment, thereby degrading input interpretation and downstream decision making.

The 'Self-Action' stage shows the highest concentration of bugs. 'API Misuse' (accounting for 32.77% of this stage) and 'API Incompatibility' (accounting for 23.92% of this stage) dominate, followed by 'Streaming Instability', 'Documentation Desync', 'Serialization Error', 'Configuration Misalignment', and 'Dependency Environment Inconsistency'. These core API-related issues occur frequently during scheduling, tool invocation, and graph execution. Improper API usage and incompatible interface specifications often lead to mismatched data flow, failed tool invocation, and broken workflow execution.

The 'Mutual Interaction' stage features coordination failures, including 'API Misuse' (accounting for 37.74% of this stage) and 'Streaming Instability' (accounting for 15.09% of this stage), which disrupt delegation logic and shared state consistency.

The 'Evolution' stage focuses on 'API Misuse' (accounting for 44.44% of this stage), ' API Incompatibility' and 'Memory Persistence Bug', where integration errors, compatibility gaps, or stale states weaken long-term continuity and adaptation.

> **Finding 1:** Root causes concentrate in the 'Self-Action' stage, which contains most reported cases. 'API Misuse' (32.97%) and 'API Incompatibility' (22.34%) together account for 55.3% of all issues and mainly arise during execution. In contrast, the 'Perception', 'Mutual Interaction', and 'Evolution' stages contain far fewer bugs, indicating the majority of bugs are associated with the execution phase.

3.2 RQ2: Symptoms

We conduct annotation and summarization on 998 issue reports in the dataset and identify the following seven core symptom categories and their distribution. Detailed results are shown in Figure 3.

- *S1.Crash (10.02%).* This symptom represents the most explicit category of an agent framework bug. The framework crashes, terminates unexpectedly, or loses its execution capability because internal components fail at runtime. The agent cannot continue task execution once the bug triggers.
- *S2.Functional Error (78.26%).* This symptom reflects framework-level logic errors that do not necessarily terminate execution. The system keeps running, but core mechanisms such as state tracking, planning flow, or tool coordination behave incorrectly. As a result, the agent produces wrong outcomes or fails to complete tasks reliably.
- *S3.Build Failure (6.71%).* This symptom indicates structural or dependency-related bugs in the framework. Installation, configuration, or startup processes break before the agent enters an operational state, which prevents any task execution from beginning.
- *S4.Poor Performance (1.00%).* This symptom reveals efficiency-related bugs in framework design or resource management. The agent continues operating, but response time increases, resource usage grows abnormally, or throughput drops, which degrades overall system effectiveness.
- *S5.Hang (1.70%).* This symptom exposes control-flow or coordination bugs inside the agent frameworks. Execution stops progressing without explicit errors, and the agent remains stuck in an unresolved state instead of completing or terminating the workflow.
- *S6.Unreported (0.40%).* This symptom refers to silent failures in which the framework accepts inputs or configurations but ignores them, fails to enforce constraints, or produces no explicit error signal.
- *S7.Display Anomaly (1.90%).* This symptom captures inconsistencies caused by a framework-level bug in logging, visualization, or interface rendering. The agent may execute internally, but the framework presents misleading or incorrect information to users.

Analysis on Agent Lifecycle Distribution. We analyze symptom manifestations across the five stages of the agent lifecycle and observe clear stage-level concentration patterns. Although each symptom reflects a different observable bug, their distribution reveals structural imbalances across stages.

In the 'Agent Initialization' stage, symptoms are mainly in the categories of 'Functional Error ' (accounting for 58.33% of this stage) and 'Build Failure ' (accounting for 16.67% of this stage). These symptoms directly reflect critical gaps in reliability and configuration correctness during the initialization process, revealing systemic vulnerabilities in the pre-execution preparation phase that affect the foundation for stable startup of subsequent workflows.

The 'Perception' stage frequently exhibits 'Functional Error' (accounting for 72.22% of this stage), 'Display Anomaly' (accounting for 16.67% of this stage), 'Crash', and 'Build Failure', indicating deficiencies in robustness and correctness as a core component of input processing and state understanding.

The 'Self-Action' stage shows the highest diversity of symptoms, including 'Functional Error' (accounting for 78.46% of this stage),

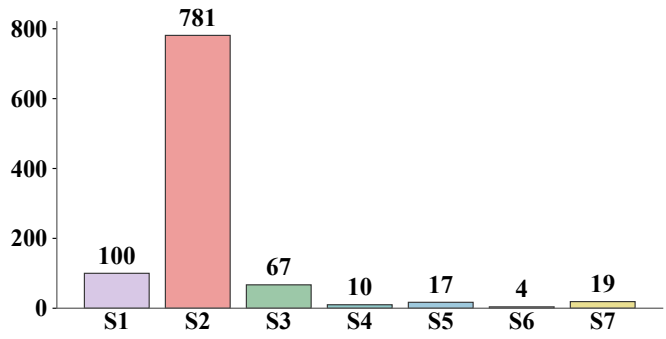


Figure 3: Symptoms Distribution of Agent Framework Bugs

'Crash' (accounting for 10.32% of this stage), 'Build Failure' (accounting for 6.58% of this stage), 'Hang', and 'Display Anomaly'. The 'Self-Action' stage, as it involves the agent autonomously executing actions and interacting with the environment, exhibits the highest symptom diversity, indicating that this phase represents the primary source of risk for the agent's execution reliability.

The 'Mutual Interaction' stage primarily involves 'Functional Error' (accounting for 92.45% of this stage), 'Crash', 'Build Failure', 'Poor Performance' and 'Display Anomaly', which weaken coordination and shared state consistency across agents.

The 'Evolution' stage mainly presents 'Functional Error' (accounting for 66.67% of this stage), 'Crash', and 'Build Failure', indicating that long-running and adaptive workflows remain susceptible to compatibility gaps and stale-state problems.

Finding 2: Bug symptoms peak in the 'Self-Action' stage, with 'Crash' (accounting for 10.32% of this stage), 'Functional Error' (accounting for 78.46% of this stage), and 'Build Failure' (accounting for 6.58% of this stage) dominating, while 'Poor Performance' and 'Unreported' occur less frequently.

4 Conclusion & Future Work

We present a lifecycle-oriented study of bugs in agent frameworks by analyzing 998 issue reports from CrewAI and LangGraph. We identify dominant root causes, observable symptoms, and their structured relationships, showing how framework-level issues affect agent execution. Our taxonomy of 15 root causes and seven symptoms across five lifecycle stages highlights API-related problems and execution-stage mechanisms as primary failure drivers. This paper provides insights for the development and maintenance of LLM agent infrastructure, promoting research on the LLM software supply chain.

This paper provides a systematic study of agent framework bugs. Future work can explore lifecycle-aware testing and analysis methods to detect execution-semantic bugs early. As agent ecosystems evolve, research should address state-consistency, coordination, and resource-management bugs in multi-agent and long-running workflows. Finally, integrating our insights into framework design, including standardized APIs, dependency management, and runtime monitoring, may help reduce systemic failures.

Acknowledgments

The authors would like to thank the anonymous reviewers for their insightful comments and valuable suggestions. This work is supported in part by the National Research Foundation, Singapore, and the Cyber Security Agency under its National Cybersecurity R&D Programme (NCRP25-P04-TAICeN). Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of the National Research Foundation, Singapore, and Cyber Security Agency of Singapore.

References

[1] 2023. https://github.com/langchain-ai/langgraph. Accessed: 2026-01-10.
[2] 2024. https://github.com/crewAIInc/crewAI. Accessed: 2026-01-10.
[3] Mert Cemri, Melissa Z Pan, Shuyi Yang, Lakshya A Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, et al. 2025. Why do multi-agent llm systems fail? *arXiv preprint arXiv:2503.13657* (2025).
[4] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Guss, Alex Nichol, Alex Paino, Nolan Tezak, Jie Tang, Shantanu Kundu, Siddharth Singh, Gábor Melis, Noah Lerner, Sam McCandlish, Erich Elsen, Natalie Scharli, Benjamin Chess, Rafal Jozefowicz, Cyprien de Masson d'Autume, Aditya Ramesh, Wojciech Zaremba, and Ilya Sutskever. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021).
[5] Xin Huang, Zhe Chen, Yuxiang Zhang, Yue Wang, and Hongsheng Li. 2023. A Survey on Large Language Models for Autonomous Driving. *arXiv preprint arXiv:2306.06066* (2023).
[6] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems* 33 (2020).
[7] Yinan Li, Lin Ma, Yanchao Wang, Kai Chen, Zhen Wang, Yu Qiao, and Ping Luo. 2023. Language-Conditioned Autonomous Driving. *arXiv preprint arXiv:2302.12969* (2023).
[8] Shenao Wang, Yanjie Zhao, Xinyi Hou, and Haoyu Wang. 2025. Large language model supply chain: A research agenda. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–46.
[9] Ziluo Xue, Yanjie Zhao, Shenao Wang, Kai Chen, and Haoyu Wang. 2025. A Characterization Study of Bugs in LLM Agent Workflow Orchestration Frameworks. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 3369–3380.
[10] Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, et al. 2025. Which agent causes task failures and when? on automated failure attribution of llm multi-agent systems. *arXiv preprint arXiv:2505.00212* (2025).

Received 12 February 2026; accepted 22 March 2026